

COMPARISON OF COMBINATORIAL AND OPTIMIZATION METHODS FOR THE SHORTEST-PATH PROBLEM

BY

JASMINE SALDANO

SENIOR THESIS

Submitted in partial satisfaction of the requirements for the degree of

BACHELOR OF SCIENCE

in

MATHEMATICS

in the

COLLEGE OF LETTERS AND SCIENCE

of the

UNIVERSITY OF CALIFORNIA, DAVIS



Approved:

Chadwick Kopy

Contents

Acknowledgements	3
Introduction	4
1 Background	5
1.1 Graph Theory	5
1.2 Combinatorial Algorithms	8
1.2.1 Dijkstra’s Algorithm	8
1.2.2 Bellman-Ford Algorithm	10
1.2.3 Comparison of Dijkstra and Bellman-Ford	12
1.3 Linear Programming and Optimization	13
2 Experimental Design	17
2.1 PassageMath Environment	18
2.2 Algorithm Implementations	18
3 Results and Discussion	23
3.1 Runtime and Scalability Comparisons	24
3.1.1 Effect of Graph Size	24
3.1.2 Effect of Edge Density	24
3.1.3 Effect of Edge-Weight Range	25
3.1.4 LP Model Construction vs. Solver Time	26
3.1.5 Overall Runtime Trends	28
Bibliography	28

Acknowledgements

I want to express my immense appreciation to Professor Matthias Köppe for this amazing opportunity and for guiding me throughout this thesis and the Math Lab. His support and ideas encouraged me to learn, ask questions, and explore more mathematics through real-world applications than I originally expected.

I am also especially grateful to Marji LeGrand, whose encouragement through her advising in the Mathematics Department helped me continue pursuing mathematics and ultimately led me to this point. I also want to thank Professor Ian Davidson, whose teaching introduced me to Dijkstra's and Bellman-Ford algorithms and who was the first to encourage me to pursue research, and Professor Stefan Mihajlović, whose teaching and encouragement furthered my interest in mathematics and who first brought the Math Lab opportunity to my attention.

Finally, I want to thank the friends and others I have met throughout my time at UC Davis, including those who may never read this, who helped guide me along the way. I am grateful to be concluding my time at UC Davis with this thesis and completing my degree.

Introduction

Graph problems arise in many areas of mathematics and computer science, particularly in combinatorics, optimization, artificial intelligence, machine learning, transportation systems, and communication networks. Among the most fundamental graph problems are shortest path, routing, and network flow, which model tasks such as GPS navigation, resource allocation, and path planning for intelligent systems. Many of these problems can be solved through either combinatorial graph algorithms or optimization-based methods.

This thesis compares the classical combinatorial algorithms Dijkstra and Bellman-Ford with a linear programming formulation through computational experiments in PassageMath. Computational comparisons between algorithms are important because algorithm performance can depend heavily on graph properties such as size, density, structure, and weighting conditions. Additionally, optimization-based methods may provide flexibility for more complex constraints, while combinatorial algorithms may mostly offer strong efficiency on classical graph problems. Through computational experiments, we analyze runtime behavior, scalability, and sensitivity to graph structure using randomly generated graphs. With this, we aim to better understand the strengths and limitations of combinatorial and optimization-based techniques on various classes of graphs.

The SageMath and PassageMath software systems provide a useful environment for this study due to their extensive graph theory, combinatorics, and optimization libraries. In particular, SageMath contains existing implementations of shortest path algorithms, network flow routines, and linear programming solvers, making it possible to compare algorithmic and optimization-based techniques within a unified computational framework.

Chapter 1

Background

Graph theory, combinatorial optimization, and linear programming are closely connected areas of mathematics and computer science that appear in many real-world computational problems. Graphs provide mathematical models for representing relationships and networks, where vertices represent objects and edges represent connections between them. The first paper on graph theory was written in 1736 by the famous Swiss mathematician Leonhard Euler, which dealt with the well-known Königsberg bridge problem [1]. Graph theory has historical roots in puzzles and games, but today it provides a natural framework for investigations in many disciplines, including networks, chemistry, psychology, social science, and genetics. Graphs are some of the most useful models in computer science, becoming especially valuable in routing, artificial intelligence, and machine learning.

1.1 Graph Theory

A graph G , also called a simple graph, is composed of two objects. It has a finite set of elements $V = \{a, b, c\}$ called vertices or nodes, and a set E of pairs of distinct vertices called edges. Every node v is usually depicted as a point in the graph, and every edge e is usually depicted as a line that connects exactly two nodes [4]. A graph whose vertex set is V and whose edge set is E is denoted by

$$G = (V, E).$$

An edge is a tuple (u, v) that represents the link between nodes u and v . The number of edges connected to a node v is called the degree of v [4]. Generally, a simple graph is assumed to have at most one edge connecting any pair of vertices, and no vertex is connected to itself [1]. In an undirected graph, all edges can be traversed in both directions, whereas an edge (u, v) of a directed graph can only be traversed from u to v [4].

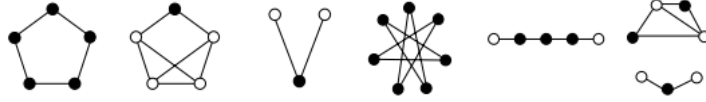


Figure 1.1: Examples of simple graphs.

In Figure 1.1 are various examples of simple graphs, where some graphs contain an odd number of nodes and black circles indicate nodes of even degree [1].

A walk in a graph G is a sequence of one or more nodes $v_0, v_1, \dots, v_k$ such that any two consecutive nodes in the sequence are adjacent. The shortest possible walk consists of a single node v_0 . If each edge is assigned a nonnegative real number, then this value represents the length or weight of the edge.

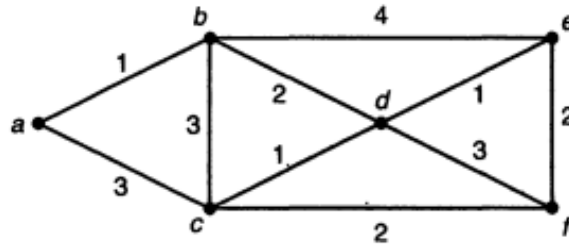


Figure 1.2: Weighted graph example.

In the weighted graph depicted in Figure 1.2, the numbers beside edges denote edge lengths, also called the weights [1]. One example of a walk joining vertices a and e is a, b, d, e , which has length 4 after adding the edge lengths.

A path $P = \{s, \dots, u, \dots, v, \dots, t\}$ in a graph is a sequence of nodes connected by edges from a source node s to a target node t [4]. Additionally, a path is a walk that does not repeat any vertices. The first and last vertices are called endpoints of the path, and connecting them forms a cycle such as in Figure 1.3. A path $P' = \{u, \dots, v\}$ is a subpath of P if its sequence of nodes and edges are all contained within the same order in P [4]. The weight of a path $w(P)$ is the sum of all the weights associated to the edges involved in the path [4].

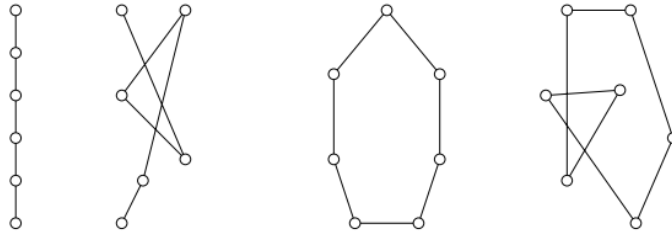


Figure 1.3: Examples of paths and cycles.

A graph G is connected if there exists a path between every pair of vertices. Graphs are either connected or disconnected, but some connected graphs are "more connected" than others [1]. This can be measured by how many vertices or edges must be removed in order to disconnect the graph. A graph that is not connected can be decomposed into connected components, also called maximal connected subgraphs [1]. Connected components capture the idea that a graph may consist of multiple independent regions of connectivity.

Theorem 1.1.1. [1] *Let $G = (V, E)$ be a graph. Then the vertex set V can be uniquely partitioned into nonempty subsets $V_1, V_2, \dots, V_k$ such that:*

1. *The induced subgraph $G_i = (V_i, E_i)$ is connected for each i .*
2. *If $i \neq j$, then there is no path in G between any vertex in V_i and any vertex in V_j .*

Each connected component acts as an independent subgraph in which standard graph problems are well-defined. In particular, vertices belonging to different components are not reachable from one another. This has direct implications for shortest path problems. If no path exists between vertices s and t , then no finite-length path can be formed between them, and the distance between such vertices is typically considered infinite or undefined. As a result, shortest path algorithms are naturally applied within connected components.

For vertices s and t within the same connected component, the shortest path between them is a path of minimum total weight among all paths connecting s to t . The corresponding shortest path distance is defined as

$$d(s, t) = \min_{P: s \rightarrow t} \sum_{e \in P} w(e),$$

where the minimum is taken over all paths P from s to t , and $w(e)$ denotes the weight of edge e . This is also called the minimum distance.

Beyond shortest path problems, graphs can also model the movement of quantities through a network. Flow problems model the movement of quantities through a directed graph, where each edge is assigned a nonnegative capacity representing the maximum allowable transfer. Unlike shortest path problems, which seek a single minimum-weight path between two vertices, flow problems distribute quantity across multiple paths between a source s and a sink t . These models extend weighted graph structures and motivate optimization-based formulations of graph problems discussed in Section 1.3.

In graphs that model physical situations, such as the paths in the maps of Figure 1.4, some edges are considered more costly than others. As such, edges are often assigned weights that represent quantities such as distance, time, cost, or energy. These weights allow graphs

to encode optimization problems over networks, where the goal is to compare and select among competing routes or patterns of connectivity.

As a result, weighted graphs are fundamental in providing the underlying structure for both shortest path and flow problems, since both rely on evaluating and optimizing paths in a network. This connection between graph structure and optimization motivates the algorithmic and linear programming approaches studied in later sections.

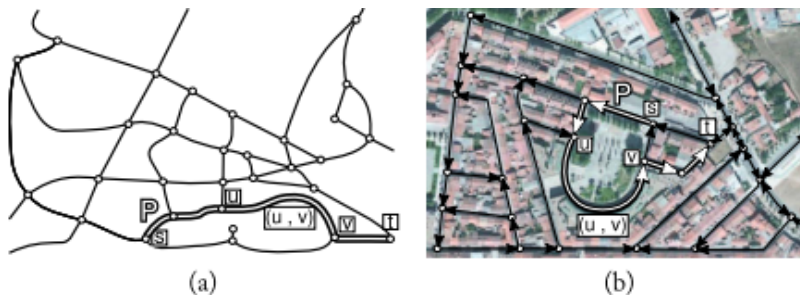


Figure 1.4: Examples of real-world paths in (a) an undirected graph and (b) a directed graph

1.2 Combinatorial Algorithms

Many graph problems can be solved using combinatorial algorithms, which operate directly on the discrete structure of a graph. These algorithms systematically explore vertices and edges according to specific rules and are widely used in computer science, optimization, and artificial intelligence.

1.2.1 Dijkstra's Algorithm

As explained in section 1.1, shortest path algorithms attempt to determine paths of minimum total cost between vertices in a weighted graph. One of the most well-known shortest path algorithms is Dijkstra's algorithm, introduced by Edsger W. Dijkstra in 1956 [4]. Dijkstra's algorithm computes shortest paths from a source node s to all of the remaining and reachable nodes in a graph with nonnegative edge weights [4]. The exploring process is called edge relaxation. Relaxing an edge (u, v) attempts to improve the current best-known distance to v [4]. Therefore, there exists a path from a source node u to reach v with a tentative shortest distance. This update step can be written as

$$d(v) = \min(d(v), d(u) + w(u, v)).$$

Since Dijkstra's algorithm requires all edge weights to be nonnegative, on a weighted directed graph $G = (V, E)$ we assume that $w(u, v) \geq 0$ for each edge $(u, v) \in E$ [4]. The

algorithm begins by assigning every vertex an initial tentative distance of ∞ , except for the source vertex s , which is assigned distance 0. A min-priority queue Q stores vertices together with their current tentative distances, as implemented in Algorithm 1.

At each iteration, the algorithm removes the vertex u with smallest tentative distance from Q . Once a vertex is extracted from the priority queue, its shortest-path distance is considered finalized under the assumption of nonnegative edge weights. The algorithm then examines all adjacent vertices v . If traversing the edge (u, v) produces a shorter path to v , the tentative distance of v is updated and the pair $(d(v), v)$ is inserted into the queue. This process is known as edge relaxation, and continues until all reachable vertices have been explored.

The running time of the algorithm is

$$O((|V| + |E|) \log |V|),$$

when implemented using a binary heap priority queue.

Algorithm 1 Dijkstra's Algorithm

```

1: Choose source vertex  $s$ 
2: Initialize  $d(v) = \infty$  for all vertices  $v$ 
3:  $d(s) = 0$ 
4: Initialize priority queue
5:  $Q = [(0, s)]$ 
6: while  $Q \neq \emptyset$  do
7:    $(d, u) = \text{Extract-Min}(Q)$ 
8:   for each neighbor  $(v, w)$  adjacent to  $u$  do
9:     if  $d(v) > d(u) + w$  then
10:       $d(v) = d(u) + w$ 
11:      Insert  $(d(v), v)$  into  $Q$ 
12:     end if
13:   end for
14: end while

```

In contrast to the textbook formulation, which separates vertices into the finalized set S and non-finalized set $Q = V - S$, this implementation uses a priority queue-based approach without explicitly maintaining this distinction [3]. The classical formulation ensures that each vertex is processed exactly once upon finalization, whereas this heap-based implementation may contain multiple entries per vertex, allowing us to implicitly ignore outdated distance estimates when extracted.

Figure 1.5 below illustrates the execution of Dijkstra's algorithm, where the source vertex s is placed on the leftmost side of the graph [3]. Each vertex is labeled with its current estimate of the shortest-path distance, and shaded edges indicate which previous vertices are used to reconstruct shortest paths.

Graph (a) depicts the initial configuration before the algorithm begins processing, where the source vertex has distance 0 and all other vertices have distance ∞ [3]. At this point, the priority queue contains only the source vertex with its initial distance.

Graphs (b)-(f) depicts the step-by-step progression of the algorithm from the while loop as vertices are removed from the priority queue in order of smallest current distance estimate. At each step, the extracted vertex u is used to relax all outgoing edges (u, v) , which is intuitively the algorithm trying to improve its current best guess for one node v using another node u . Here, it is updating distance estimates when a shorter path is found and re-inserting updated entries into the priority queue again. Over time, the algorithm is essentially making distance estimates between nodes more accurate over time, with each "relaxation." The final subfigure (f) shows the resulting shortest-path distances and constructed parent-child structure after all reachable vertices have been processed or no shorter updates remain [3].

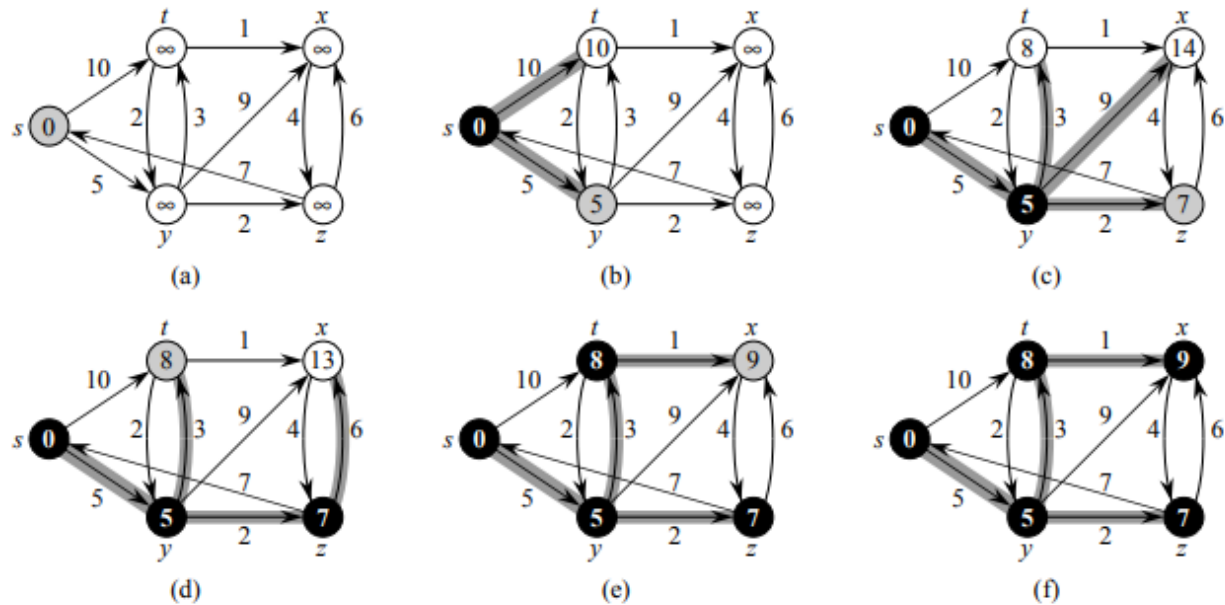


Figure 1.5: Dijkstra's algorithm [3]

1.2.2 Bellman-Ford Algorithm

Unlike Dijkstra's algorithm, the Bellman-Ford algorithm can handle graphs containing negative edge weights. Rather than greedily selecting the nearest unexplored vertex, Bellman-

Ford repeatedly relaxes all edges until shortest distances stabilize.

Given a weighted, directed graph $G = (V, E)$ with source node s and weight function $w : E \rightarrow \mathbb{R}$, the Bellman-Ford algorithm returns a boolean value indicating whether or not there is a negative-weight cycle that is reachable from the source [3]. If there is such a cycle, the algorithm indicates that no solution exists. If there is no such cycle, the algorithm produces the shortest paths and their weights [3].

The algorithm initializes all distance estimates to ∞ , except for the source vertex s , which is assigned distance 0. It then repeatedly relaxes all edges in the graph, where relaxing an edge (u, v) once again updates the tentative distance of v if a shorter path through u is found [3]. This relaxation step can be expressed as

$$d(v) = \min(d(v), d(u) + w(u, v)).$$

In the implementation used in this work (Algorithm 2), all edges are relaxed at most $|V| - 1$ times because any shortest path in a graph without cycles contains at most $|V| - 1$ edges [3]. To improve efficiency, the algorithm terminates early if no updates occur during an iteration [3].

Algorithm 2 Bellman-Ford Algorithm

```

1: Choose source vertex  $s$ 
2: Initialize  $d(v) = \infty$  for all vertices  $v$ 
3:  $d(s) = 0$ 
4: for  $i = 1$  to  $|V| - 1$  do
5:   updated = false
6:   for each edge  $(u, v) \in E$  do
7:     if  $d(u) + w(u, v) < d(v)$  then
8:        $d(v) = d(u) + w(u, v)$ 
9:       updated = true
10:    end if
11:  end for
12:  if updated = false then
13:    break
14:  end if
15: end for
    return  $d$ 

```

The running time of the algorithm is

$$O(|V||E|),$$

since each of the at most $|V| - 1$ iterations scans all edges once.

Figure 1.6 illustrates the execution of the Bellman-Ford algorithm on a directed weighted graph [3]. The source vertex s is initialized with distance 0, while all other vertices are assigned initial distance ∞ .

Graphs (a)–(e) show the state of the algorithm after successive passes over all edges. Unlike Dijkstra’s algorithm, which uses a priority queue to greedily select the next vertex with minimum tentative distance, the Bellman-Ford algorithm performs global relaxations over all edges in each iteration [3]. This means that instead of focusing on one vertex at a time, it systematically updates all distance estimates in parallel-like iterations. As such, vertices are not permanently selected or removed [3].

Because of this structure, Bellman-Ford does not require a greedy selection structure, such as a priority queue, and can correctly handle negative edge weights. In contrast, negative edge weights invalidate the greedy selection strategy used in Dijkstra’s algorithm [3].

In the final iteration (f), no further distance updates occur, indicating that the shortest-path estimates have stabilized [3]. In the implementation used here, the algorithm terminates early if a full pass produces no changes, since no further improvements are possible.

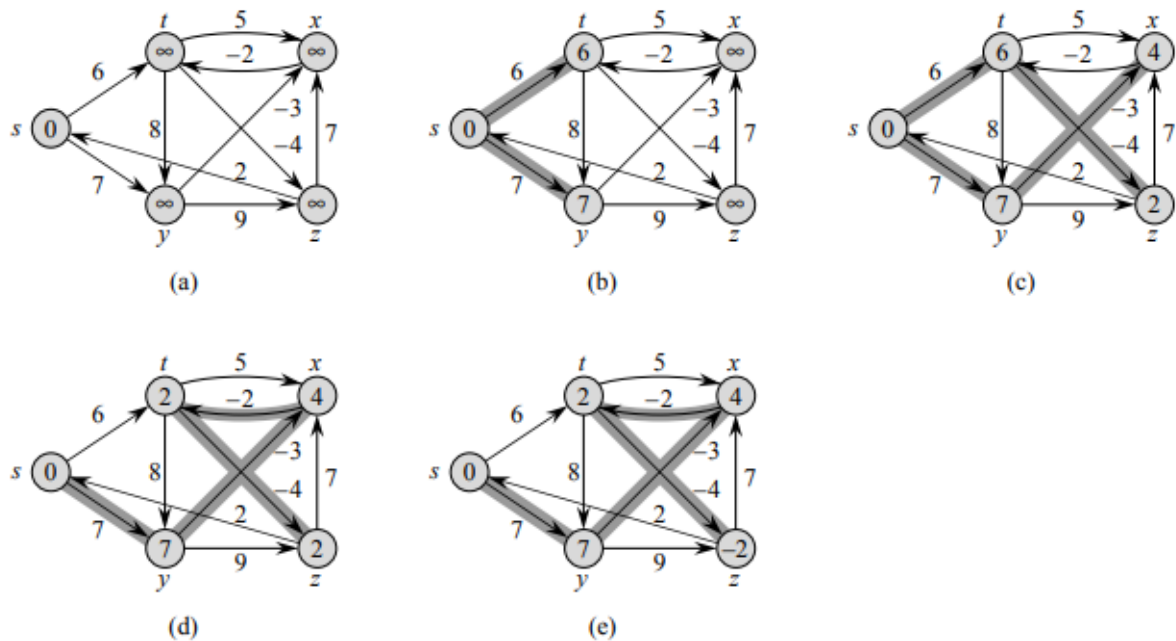


Figure 1.6: Bellman-Ford algorithm [3]

1.2.3 Comparison of Dijkstra and Bellman-Ford

Although both Dijkstra’s and Bellman-Ford algorithms solve the single-source shortest path problem, they differ significantly in strategy and applicability. Dijkstra’s algorithm uses a

greedy vertex-selection process and is generally more efficient, but it requires all edge weights to be nonnegative. In contrast, the Bellman-Ford algorithm repeatedly relaxes all edges without relying on a greedy selection structure, allowing it to correctly handle negative edge weights and detect negative-weight cycles. These differences illustrate the tradeoff between computational efficiency and generality in shortest path optimization problems.

1.3 Linear Programming and Optimization

Linear programming was a term introduced in the 1950s that originally referred to military plans or schedules for training, logistical supply, or deployment of men when computers were not commonly used [5]. Eventually, it started to be used for planning all sorts of economic activities, including transporting raw materials and products.

Despite what the name suggests, linear programming is not directly related to computer programming. The word linear refers to feasible plans restricted by linear constraints (inequalities), and that the quality of the plan (i.e., costs or duration) is measured by a linear function of the qualities being considered [5]. An example of a very simple linear programming problem (or linear program for short) is the following:

Maximize $x_1 + x_2$ subject to the constraints

$$x_1 \geq 0$$

$$x_2 \geq 0$$

$$x_2 - x_1 \leq 1$$

$$x_1 + 6x_2 \leq 15$$

$$4x_1 - x_2 \leq 10.$$

Each inequality defines a half-plane in $\mathbb{R}^2$, and the set of all points satisfying the constraints forms a convex polygon [5].

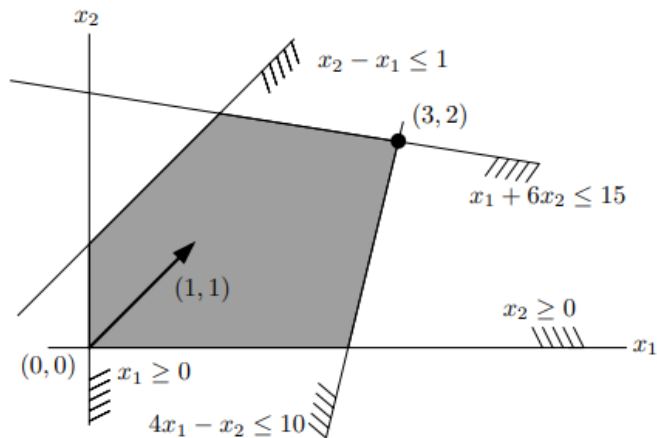


Figure 1.7: Polygon region [5]

The objective function $x_1 + x_2$ defines a family of parallel lines $x_1 + x_2 = c$, where larger values of c correspond to lines "farthest in the direction" of the vector $(1,1)$ [5]. Since the objective function is constant along lines perpendicular to $(1,1)$, translating these lines in the direction of the vector increases the objective value. The optimal solution is therefore the last point where one of these moving lines still intersects the feasible region while satisfying all constraints.

In this example, the optimal solution occurs at $(3,2)$, giving the maximum value

$$x_1 + x_2 = 3 + 2 = 5$$

. See Figure 1.8 for this illustration.

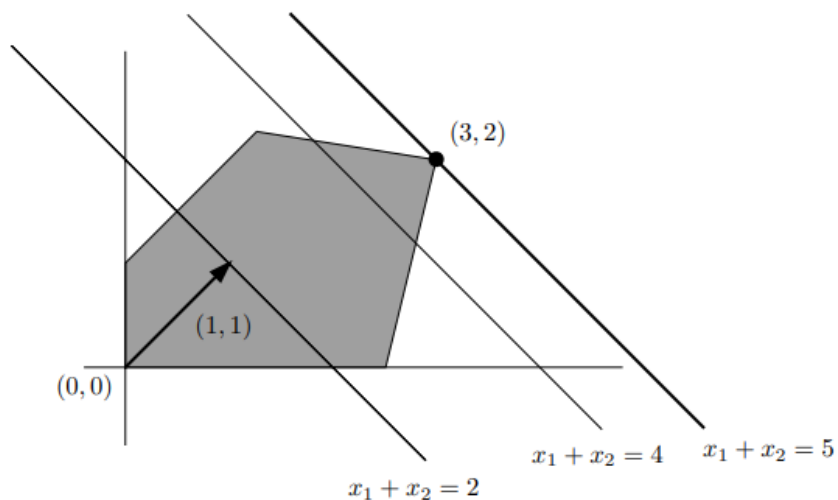


Figure 1.8: Moving lines[5]

In general, a linear program seeks to find a vector $x^* \in \mathbb{R}^n$ that maximizes or minimizes the value of a linear objective function subject to a system of linear constraints [5]. A standard linear programming formulation can be written as

$$c^T x = c_1 x_1 + \cdots + c_n x_n$$

where $c \in \mathbb{R}^n$ is a given vector. The linear equations and inequalities in the linear program are known as the constraints, and the number of constraints can be denoted by m [5]. Since a linear program is often written using matrices and vectors similar to the notation $Ax = b$, where A is a given $m \times n$ matrix and $c \in \mathbb{R}^n$ and $b \in \mathbb{R}^m$ are vectors, each linear program can be written as

$$\text{maximize } c^T x$$

$$\text{satisfying } Ax \leq b,$$

Any vector satisfying the constraints is called a feasible solution, while a solution that optimizes the objective function is called an optimal solution [5].

Unlike combinatorial algorithms such as Dijkstra's and Bellman–Ford, which are designed as fixed procedures with greedy update steps for specific graph problems, linear programming approaches the shortest-path problem by providing a general framework for modeling global optimization problems. Instead of prescribing a single step-by-step procedure, a linear program encodes a problem as a set of linear constraints and an objective function, and the solution is obtained through a general-purpose optimization method.

In this sense, combinatorial algorithms directly exploit graph structure to produce solutions with well-defined computational complexity. They construct a path incrementally by exploring vertices and edges, whereas linear programming searches over a continuous feasible region defined by linear constraints simultaneously. The performance of a linear programming solution therefore depends on the structure of the instance and the chosen solver, rather than a single fixed procedure.

A particularly important class of linear programs arises in network flow problems, where the variables represent flows along edges of a graph. In this setting, the linear constraints naturally encode conservation of flow at each node, making linear programming a natural framework for studying routing and shortest-path problems.

In a network flow formulation, each directed edge $(u, v) \in E$ is assigned a variable x_{uv} representing the amount of flow sent from node u to node v [6]. Each edge is also associated with a cost c_{uv} , representing the cost per unit of flow along that edge [6]. The objective is

to minimize the total cost of routing flow through the network:

$$\min \sum_{(u,v) \in E} c_{uv} x_{uv}.$$

To define the constraints, we first describe the total flow entering and leaving a node $v \in V$. The total flow into node v is given by

$$\sum_{(u,v) \in E} x_{uv},$$

while the total flow out of node v is given by

$$\sum_{(v,w) \in E} x_{vw}.$$

Flow conservation requires that at each node in the network, the total incoming flow equals the total outgoing flow. This ensures that flow is neither created nor destroyed at all other vertices, but rather transported through the network from source to destination. At the source and sink, the net imbalance is fixed to enforce a unit flow from s to t [6].

$$\sum_{(u,v) \in E} x_{uv} - \sum_{(v,u) \in E} x_{vu} = \begin{cases} 1 & \text{if } v = s \\ -1 & \text{if } v = t \\ 0 & \text{otherwise} \end{cases}$$

Finally, flow along each edge must be nonnegative:

$$x_{uv} \geq 0 \quad \forall (u,v) \in E,$$

since negative flow would contradict the directed nature of the edges. This could be called the nonnegativity constraints.

Chapter 2

Experimental Design

The following section presents the experimental methodology used to compare combinatorial shortest-path algorithms with a linear programming formulation of the shortest-path problem. The experiments were designed to evaluate differences in runtime performance, computational cost, and scalability across varying graph sizes.

All experiments were conducted within a Github-based PassageMath workspace as part of the UC Davis Math Lab undergraduate research program in polyhedral geometry and optimization. The work was completed as part of an undergraduate thesis project. All final runtime experiments reported in this thesis were performed on the same desktop computer with an AMD Ryzen 7 7800X3D 8-Core processor.

Within this environment, simplified Python implementations of Dijkstra’s and Bellman-Ford algorithms were developed for direct experimental comparison and runtime analysis. These implementations operated on weighted directed graphs represented as adjacency lists and explicitly performed the relaxation procedures described in Section 1.

For the optimization-based approach, the shortest-path problem was formulated as a linear program using PassageMath’s `MixedIntegerLinearProgram` framework. Flow conservation constraints were constructed directly, while the objective function was assigned prior to each solve. PassageMath’s optimization backends were then used to solve the resulting model: HiGHS, GLPK, and CVXPY. These were tested to compare how backend choice affected computational performance. Runtime measurements were averaged across repeated executions to reduce the influence of individual timing fluctuations, with primary comparisons measuring only solver execution time. A separate experiment examined LP construction and solve times. Randomly generated weighted directed graphs were used to evaluate the effects of graph size, edge density, and edge-weight range.

2.1 PassageMath Environment

The computational experiments were conducted in a Git-based PassageMath workspace using Python. PassageMath was used primarily for the optimization-based formulation of the shortest-path problem, while Dijkstra’s and Bellman-Ford algorithms needed to be implemented directly in Python because they are only mentioned in PassageMath’s documentation and not provided as built-in methods. All three approaches were therefore incorporated into the same workspace repository and evaluated within a common computational environment.

The graphs used throughout the experiments were represented as weighted directed graphs using adjacency lists. For a graph $G = (V, E)$, each vertex $u \in V$ was associated with a list of outgoing edges of the form (v, w) , where v is the destination vertex and w is the corresponding edge weight. This representation was used for the algorithm implementations and allowed randomly generated graphs to be passed to each method.

The optimization model defines a nonnegative variable x_{uv} for each directed edge (u, v) , representing the amount of flow sent along that edge. Flow conservation constraints ensure that one unit of flow is sent from the source vertex to the target vertex, while the objective function minimizes the total weighted flow. The resulting models were solved using three available optimization backends: HiGHS, GLPK, and CVXPY.

The experimental code was organized into separate modules for the algorithm implementations, graph generation, runtime measurement, and experiment execution. The complete source code used to generate the experimental results is maintained in the project’s GitHub repository for reproducibility (<https://github.com/JasLaras/passagemath-workspace>).

2.2 Algorithm Implementations

Three approaches to the shortest-path problem were implemented for comparison: Dijkstra’s algorithm, the Bellman-Ford algorithm, and a linear programming formulation.

Dijkstra’s algorithm was implemented in Python using the `heapq` module as the priority queue. The implementation maintains a dictionary of tentative distances and updates the priority queue whenever a shorter distance is found. The central update operation corresponds directly to the relaxation step described in Section 1.1.

```
import heapq

def run_dijkstra(G):
    start = 0
    target = max(G.keys())
```

```

dist = {node: float('inf') for node in G}
dist[start] = 0

pq = [(0, start)]

while pq:
    d, u = heapq.heappop(pq)

    if u == target:
        return d

    # Relaxation (update) step
    for v, w in G[u]:
        if dist[v] > d + w:
            dist[v] = d + w
            heapq.heappush(pq, (dist[v], v))

return dist[target]

```

The Bellman-Ford implementation uses the same adjacency-list graph representation. Rather than using a priority queue, the implementation iterates through every vertex and its outgoing edges during each pass.

```

def run_bellman_ford(G):
    start = 0
    target = max(G.keys())

    dist = {node: float('inf') for node in G}
    dist[start] = 0

    nodes = list(G.keys())

    # relax edges |V| - 1 times
    for _ in range(len(nodes) - 1):
        updated = False

        # Relaxation (update) step

```

```

    for u in G:
        for v, w in G[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                updated = True

    if not updated:
        break

    return dist[target]

```

The loop is limited to $|V| - 1$ passes and includes an early termination condition when a complete pass produces no distance updates. Therefore, this implementation does not detect negative cycles as the standard Bellman-Ford algorithm typically does.

The linear programming implementation below translates the shortest-path formulation from Section 1.3 into a `MixedIntegerLinearProgram` model. The implementation was divided into two parts. The `build_lp` function constructs the optimization model and flow conservation constraints once, while the `solve_lp` function assigns the objective function using the graph's edge weights and invokes the selected optimization backend. The separation between model construction and solver execution made it possible to compare solver performance independently from LP construction overhead.

```

import time
from sage.numerical.mip import MixedIntegerLinearProgram

def build_lp(G, solver=None):

    source = 0
    target = max(G.keys())

    # LP construction
    p = MixedIntegerLinearProgram(maximization=False, solver=solver)
    x = p.new_variable(nonnegative=True)

    # Flow conservation constraints
    for node in G:
        inflow = sum(
            x[u, node]

```

```

        for u in G
        for v, w in G[u]
        if v == node
    )

    outflow = sum(
        x[node, v]
        for v, w in G[node]
    )

    if node == source:
        p.add_constraint(outflow - inflow == 1)
    elif node == target:
        p.add_constraint(inflow - outflow == 1)
    else:
        p.add_constraint(inflow == outflow)

    return p, x

def solve_lp(p, x, G):
    # Objective function
    p.set_objective(
        sum(
            w * x[u, v]
            for u in G
            for v, w in G[u]
        )
    )

    solve_start = time.perf_counter()
    p.solve()
    solve_end = time.perf_counter()

    objective = p.get_objective_value()
    solve_time = solve_end - solve_start

    return objective, solve_time

```

The resulting model was solved using the HiGHS, GLPK, and CVXPY optimization backends. All three approaches, Dijkstra, Bellman-Ford, and LP, used the same source and target convention: the source was vertex 0 and the target was the vertex with the largest label in the generated graph.

Three utility files were used to support the experiments. The `graph_generator.py` file generates random weighted directed graphs based on a specified graph size, edge probability, and maximum edge weight. The `timing.py` file measures the execution time of a given function using `time.perf_counter()`. Lastly, `benchmark.py` measures an algorithm's execution time and returns the average runtime across 10 runs.

Chapter 3

Results and Discussion

The experiments were designed to compare differences in computational cost between Dijkstra’s algorithm, Bellman-Ford, and the linear programming formulation rather than to obtain precise benchmarks. The primary goal was to evaluate how the relative runtime of each approach changes with three graph characteristics: graph size, edge density, and edge-weight range. For each experiment, one characteristic was varied while the remaining parameters were held fixed.

Each experiment generated random weighted directed graphs and evaluated all three approaches on the same graph instances. The random number generator was initialized with a fixed seed, `random.seed(42)`, to ensure reproducibility and consistent input generation. This allowed runtime difference to be compared without introducing unintended differences in the generated graphs.

Table 3.1: Experimental variables and controls

Experiment	Variable varied	Variables held fixed
Graph size	Number of vertices: 10, 20, 50, 100	Edge density = 0.3, weight range: 1–10
Edge density	Edge density: 0.1, 0.3, 0.5, 0.8	100 vertices, weight range: 1–10
Weight range	Maximum weight: 10, 100, 1000	100 vertices, edge density = 0.3

Each method was executed 10 times on each graph instance, and the resulting runtimes were averaged. The purpose of these repeated runs was to reduce the effects of variability in individual timing measurements, including potential differences between the first execution and subsequent executions.

3.1 Runtime and Scalability Comparisons

3.1.1 Effect of Graph Size

The first experiment evaluated how runtime changed as the number of vertices increased. Graphs with 10, 20, 50, and 100 vertices were generated with a fixed edge probability of 0.3 and edge weights ranging from 1 to 10. Each method was executed multiple times on the same graph instance, and the average runtime was recorded in seconds. The results are shown in Table 3.2.

Table 3.2: Runtime as a function of graph size

Vertices	Dijkstra	Bellman–Ford	HiGHS	GLPK	CVXPY
10	0.00000479	0.00000718	0.000237	0.0000270	0.01298
20	0.0000109	0.0000301	0.000191	0.000135	0.04703
50	0.0000452	0.000134	0.000373	0.000611	0.27707
100	0.000105	0.000543	0.000853	0.003843	1.28513

Dijkstra’s algorithm had the lowest runtime across all graph sizes tested. Bellman-Ford was also consistently faster than the LP approaches, although the difference between Bellman-Ford and HiGHS became relatively small at the largest graph size. At 100 vertices, Dijkstra required approximately 0.000105 seconds, Bellman-Ford required 0.000543 seconds, and HiGHS required 0.000853 seconds.

The LP results also varied substantially depending on the solver. HiGHS produced the lowest LP runtime for the larger graphs, while GLPK was the opposite and became increasingly slower as graph size increased. CVXPY exhibited substantially higher runtime than either HiGHS or GLPK and was especially sensitive to increasing graph size, reaching approximately 1.285 seconds at 100 vertices. These results demonstrate that the computational performance of the LP formulation depends strongly on the optimization backend used.

3.1.2 Effect of Edge Density

The second experiment examined the effect of edge density on runtime. The edge probability was varied from 0.1 to 0.8 with a fixed number of 100 vertices and edge weights ranging from 1 to 10. The results are shown in Table 3.3.

Table 3.3: Runtime as a function of edge density

Density	Dijkstra	Bellman–Ford	HiGHS	GLPK	CVXPY
0.1	0.0000548	0.000280	0.000633	0.001711	0.40912
0.3	0.0000500	0.000385	0.001063	0.003380	1.28554
0.5	0.0000731	0.000599	0.001298	0.006447	2.31030
0.8	0.000214	0.000671	0.002233	0.007855	4.32516

As edge density increased, the runtime of all three LP solvers generally increased, with the effect being particularly pronounced for CVXPY. Its runtime increased from approximately 0.409 seconds at a density of 0.1 to approximately 4.325 seconds at a density of 0.8. GLPK increased from approximately 0.00171 to 0.00786 seconds, while HiGHS increased from approximately 0.000633 to 0.00223 seconds.

The combinatorial algorithms remained substantially faster, although Bellman-Ford also increased consistently as density increased. Dijkstra remained comparatively stable at lower densities before increasing at the highest density tested. These trends are consistent with the increasing number of edges that must be processed as graph density increases.

3.1.3 Effect of Edge-Weight Range

The third experiment examined the effect of the numerical range of edge weights. The maximum edge weight was varied among 10, 100, and 1000 with a fixed number of 100 vertices and edge probability of 0.3. The results are shown in Table 3.4.

Table 3.4: Runtime as a function of edge-weight range

Weight Range	Dijkstra	Bellman–Ford	HiGHS	GLPK	CVXPY
10	0.000190	0.000363	0.000933	0.003972	1.26671
100	0.000130	0.000482	0.001032	0.004121	1.30904
1000	0.0000356	0.001104	0.000963	0.005917	1.30889

Unlike graph size and edge density, increasing the edge-weight range did not produce a consistent increase in runtime across all methods. The LP solver runtimes remained relatively stable compared to the changes observed in the graph-size and density experiments, particularly for HiGHS and CVXPY. GLPK showed a slight increase as the weight range increased.

Interestingly, at the largest maximum edge weight of 1000, HiGHS required approximately 0.000963 seconds, compared with approximately 0.001104 seconds for Bellman-Ford. Thus, although Dijkstra remained the fastest method, the HiGHS LP solver slightly outperformed Bellman-Ford for this graph instance.

This result suggests that under certain conditions, the LP approach may become comparatively competitive with or even outperform specialized shortest-path algorithms. For example, testing even larger edge-weight ranges beyond those tested here could provide a useful direction for further investigation into when an LP formulation may be more noticeably competitive with Bellman-Ford.

Although the difference is small, these values represent average runtime across 10 runs, and individual runs may show greater variation between the two methods. The fact that HiGHS slightly outperformed Bellman-Ford on average is therefore still worth noting.

3.1.4 LP Model Construction vs. Solver Time

The initial runtime experiments measured the complete LP process, including both model construction and solver execution. These results showed substantial runtime differences between the LP approaches and the shortest-path algorithms, motivating a closer investigation into where the computational cost of the LP approach occurred.

Unlike Dijkstra’s and Bellman-Ford, which operate directly on the graph representation, the LP approach must first construct an optimization model containing decision variables, an objective function, and flow conservation constraints before invoking the solver. Therefore, the total runtime of the LP approach can be decomposed as

$$T_{\text{LP, total}} = T_{\text{construction}} + T_{\text{solve}}.$$

To distinguish these two components, the LP implementation was modified to measure the time required to construct the model separately from the time required to solve it. This experiment varied only the graph size, using graphs with 10, 20, 50, and 100 vertices. The edge probability was fixed at 0.3, and edge weights ranged from 1 to 10. For each graph size, the same randomly generated graph was used for five independent LP runs, and the average construction, solver, and total runtimes were recorded. The results are shown in Table 3.5.

Table 3.5: LP model construction and solver runtimes				
Vertices	Solver	Construction (s)	Solve (s)	Total (s)
10	HiGHS	0.011732	0.001133	0.012865
	GLPK	0.001121	0.000090	0.001210
	CVXPY	0.136996	0.014489	0.151486
20	HiGHS	0.001688	0.000952	0.002640
	GLPK	0.000874	0.000147	0.001022
	CVXPY	0.045860	0.064036	0.109896
50	HiGHS	0.007316	0.002239	0.009554
	GLPK	0.006030	0.000798	0.006828
	CVXPY	0.890152	0.318717	1.208870
100	HiGHS	0.037225	0.007139	0.044365
	GLPK	0.028062	0.004236	0.032298
	CVXPY	13.189667	1.520120	14.709787

The results show that model construction accounted for a substantial portion of the total LP runtime for all three approaches, particularly as graph size increased. At 100 vertices, HiGHS required about 0.0372 seconds for construction and 0.00714 seconds for solving, while GLPK required about 0.0281 seconds for construction and 0.00424 seconds for solving. For these two solvers, model construction accounted for approximately 84% and 87% of their total runtimes.

The difference was especially pronounced for CVXPY at 100 vertices, where model construction required about 13.19 seconds compared with about 1.52 seconds for solving. Construction therefore accounted for approximately 90% of its total runtime. Although construction did not dominate every individual case, these results indicate that a large portion of the computational cost of the LP approach can occur during construction, before the optimization problem is actually solved.

The results also demonstrate that LP performance depends considerably on the solver and modeling backend. Although all three approaches solve the same underlying shortest-path formulation, their construction and solver runtimes differed substantially. This suggests that the higher runtime of an LP approach cannot be attributed solely to solving the optimization problem itself, since the process of constructing and representing the model can also contribute to its computational cost.

3.1.5 Overall Runtime Trends

Across the experiments, Dijkstra’s algorithm generally produced the lowest runtime, while the performance of the LP formulation varied considerably depending on the solver used. HiGHS was consistently the fastest of the LP approaches for the larger graph sizes and higher edge densities, while CVXPY had substantially greater runtimes than both HiGHS and GLPK across all tested conditions. Graph size and edge density produced the clearest increases in runtime, particularly for the LP approaches, while changes in the edge-weight range produced less consistent effects.

Although the specialized algorithms were generally faster, the results also showed that this performance difference was not the same under every tested condition. At the largest edge-weight range, HiGHS slightly outperformed Bellman-Ford on average, suggesting that there may be conditions under which the performance gap between LP and specialized shortest-path algorithms narrows or even reverses. That difference is worth investigating in further testing. However, because the difference was relatively small, it may also reflect normal runtime variability or other system-level effects rather than a consistent performance advantage. Either way, testing additional weight ranges and larger graph instances would help determine whether this pattern persists.

Overall, the experiments demonstrate a computational tradeoff between specialized shortest-path algorithms and a more general optimization-based approach. Dijkstra and Bellman-Ford generally achieved lower runtimes for direct shortest-path computation, while the LP formulation provides a more flexible optimization framework whose computational performance depends strongly on its implementation. The results also suggest that the relative advantage of specialized algorithms is not necessarily uniform across all problem characteristics, providing several directions for further investigation.

Bibliography

- [1] R. A. Brualdi, *Introductory Combinatorics*, 5th edition, Pearson Prentice Hall, 2009.
- [2] L. Lovász, J. Pelikán, K. Vesztergombi, *Discrete Mathematics: Elementary and Beyond*, 2nd edition, Springer, 2019.
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 3rd edition, MIT Press, 2009.
- [4] H. Ortega-Arranz, D. R. Llanos, A. Gonzalez-Escribano, *The Shortest-Path Problem*, Synthesis Lectures on Theoretical Computer Science, Springer, Cham, 2015. doi:10.1007/978-3-031-02574-7
- [5] J. Matoušek, B. Gärtner, *Understanding and Using Linear Programming*, Springer, Berlin, 2007.
- [6] R. J. Vanderbei, *Linear Programming: Foundations and Extensions*, 4th edition, Springer, New York, 2014.